

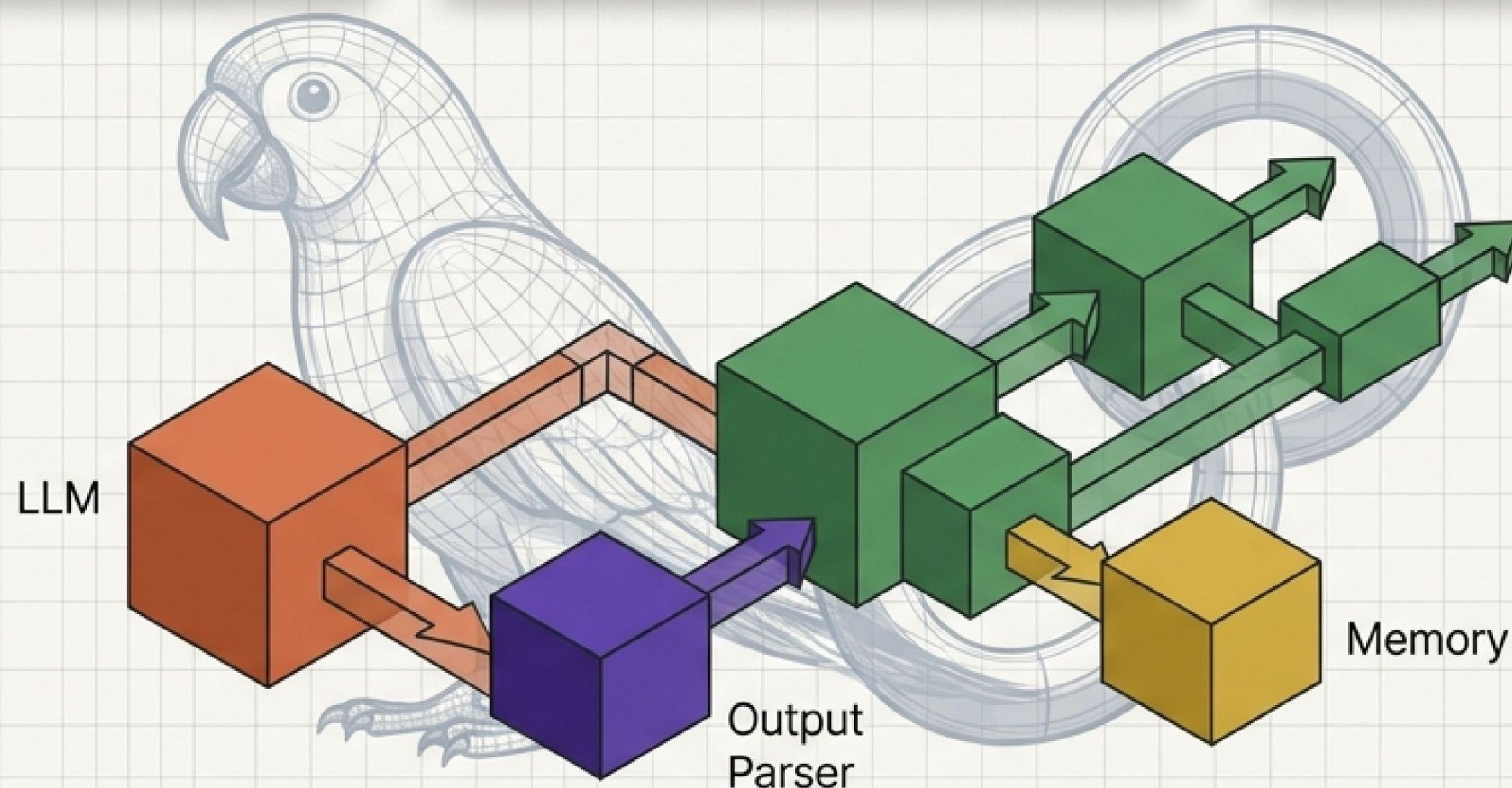
LangChain: Dal Modello all'Applicazione

Corso: NLP & Generative AI con modelli open source (Lezione 5 di 6)

> Dal prompt manuale a componenti riutilizzabili e componibili

> Orchestrazione con LangChain + Qwen2.5 su GPU T4

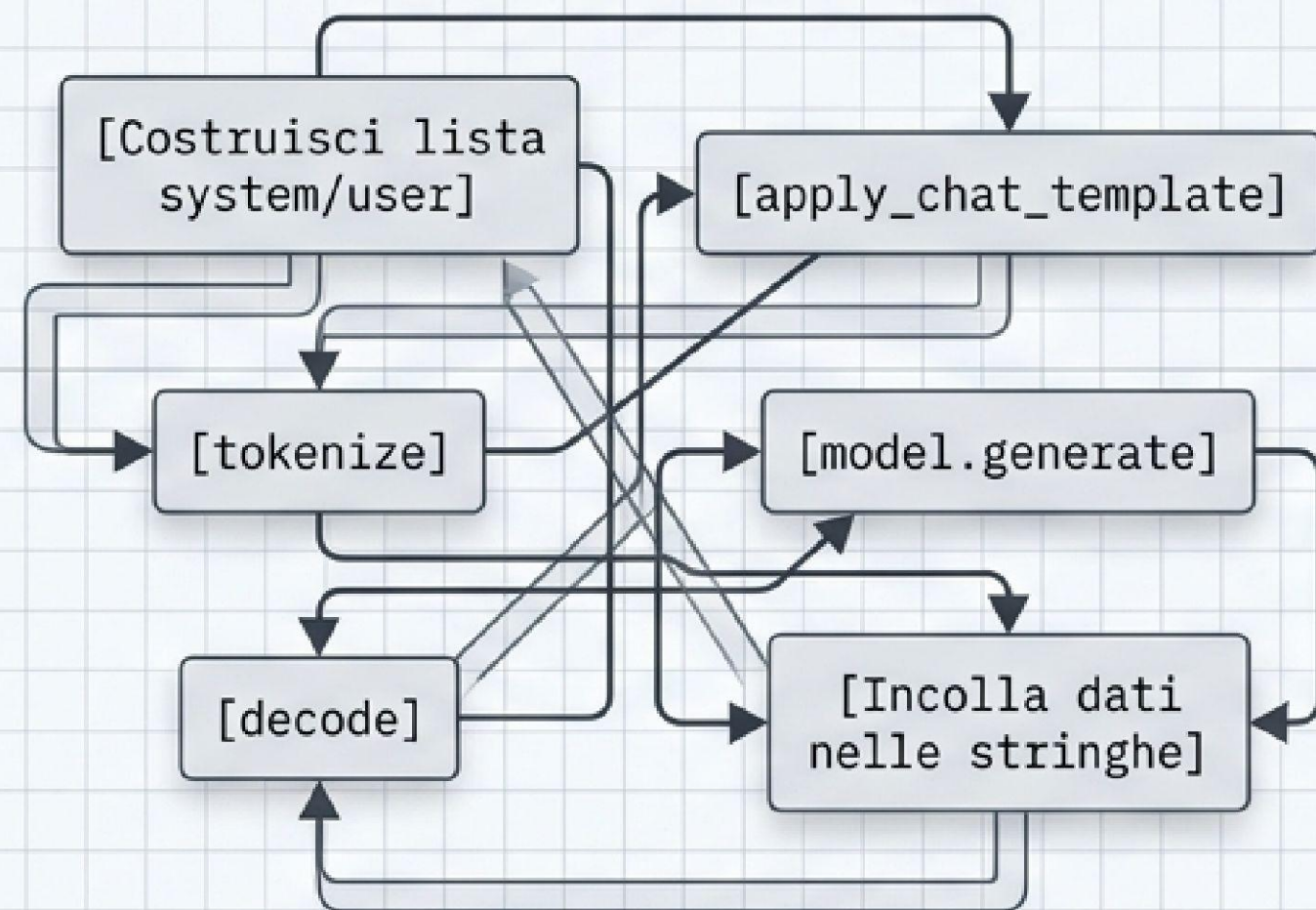
> Dominio: Analisi di recensioni clienti in italiano



Il Caos del Codice Manuale (Da Dove Veniamo)

Nella Lezione 4, usare l'LLM richiedeva troppa infrastruttura manuale.

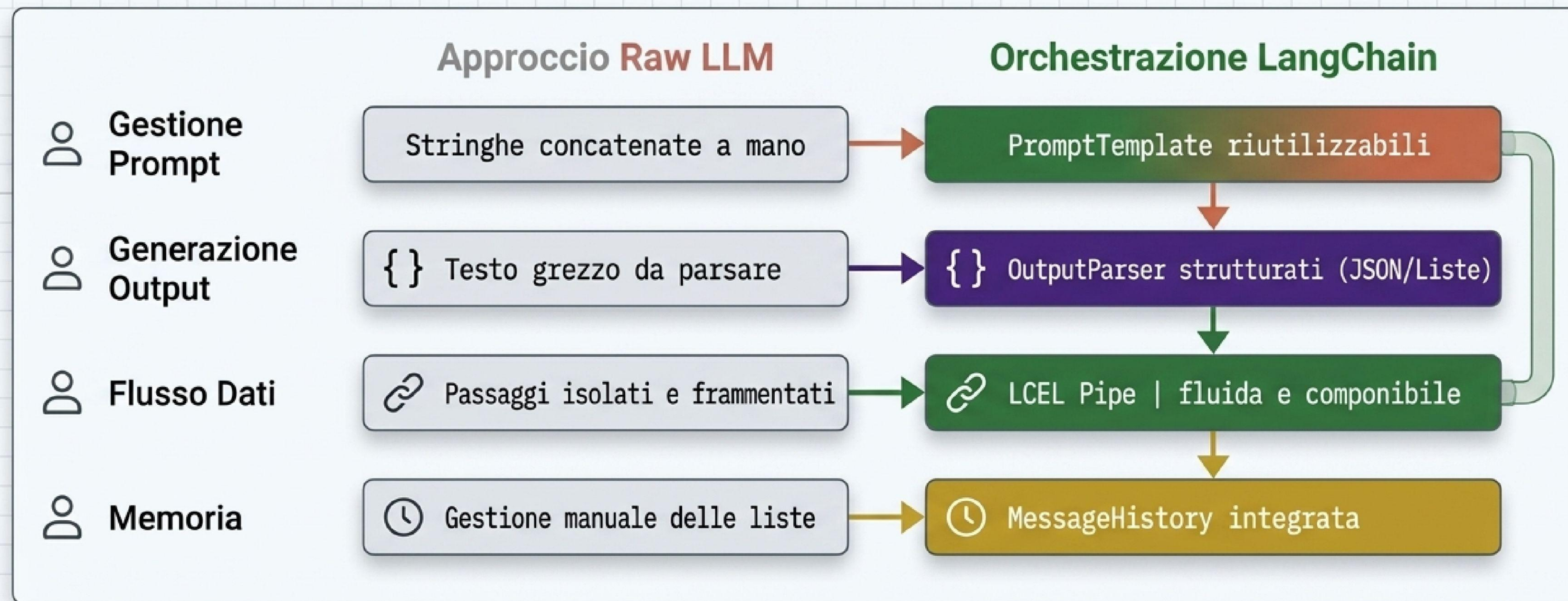
The Chaos Board



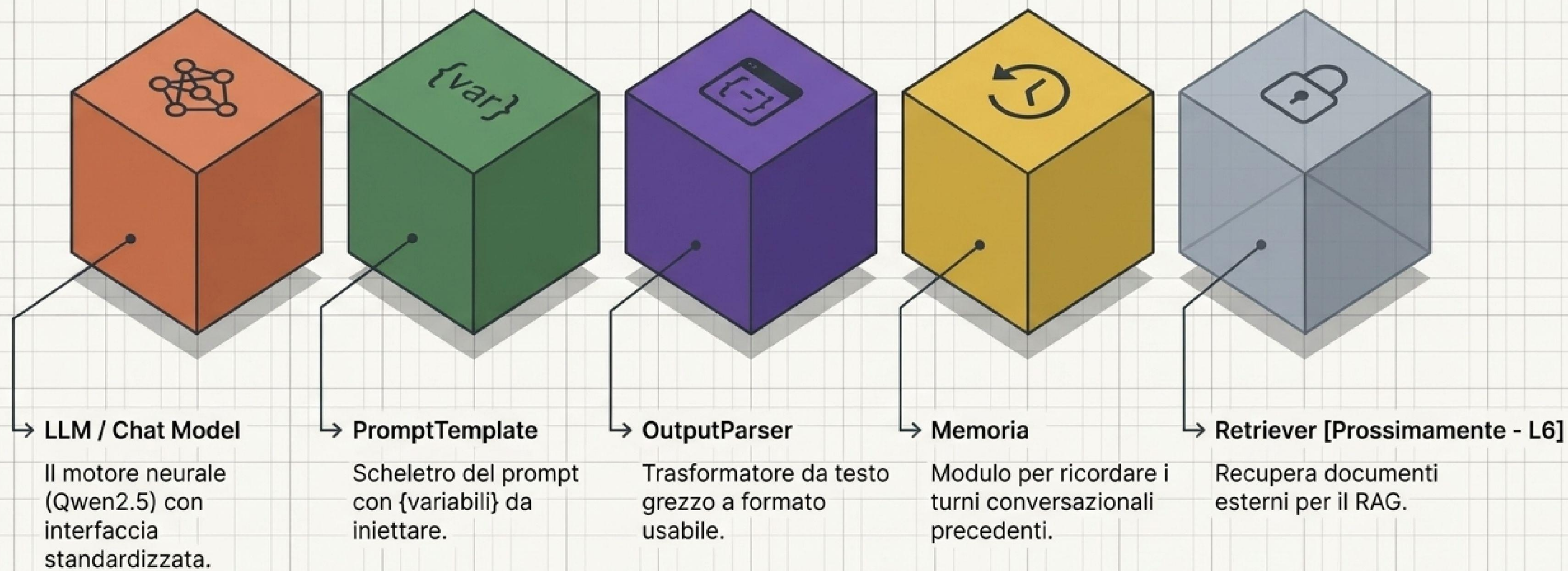
➔ Funziona per un test, ma in produzione serve un'architettura strutturata.

Che Cos'è LangChain? 🦜🔗

Il framework per orchestrare gli LLM. Non sostituisce il modello: lo avvolge e lo rende componibile. Meno codice idraulico, più logica applicativa.



I Mattoni Fondamentali



LCEL: Comporre l'Architettura con la Pipe |

LangChain Expression Language (LCEL). Colleghiamo i componenti con l'operatore |, come nelle pipe Unix. L'output di un nodo diventa l'input del successivo.

```
catena = prompt | chat | parser
```

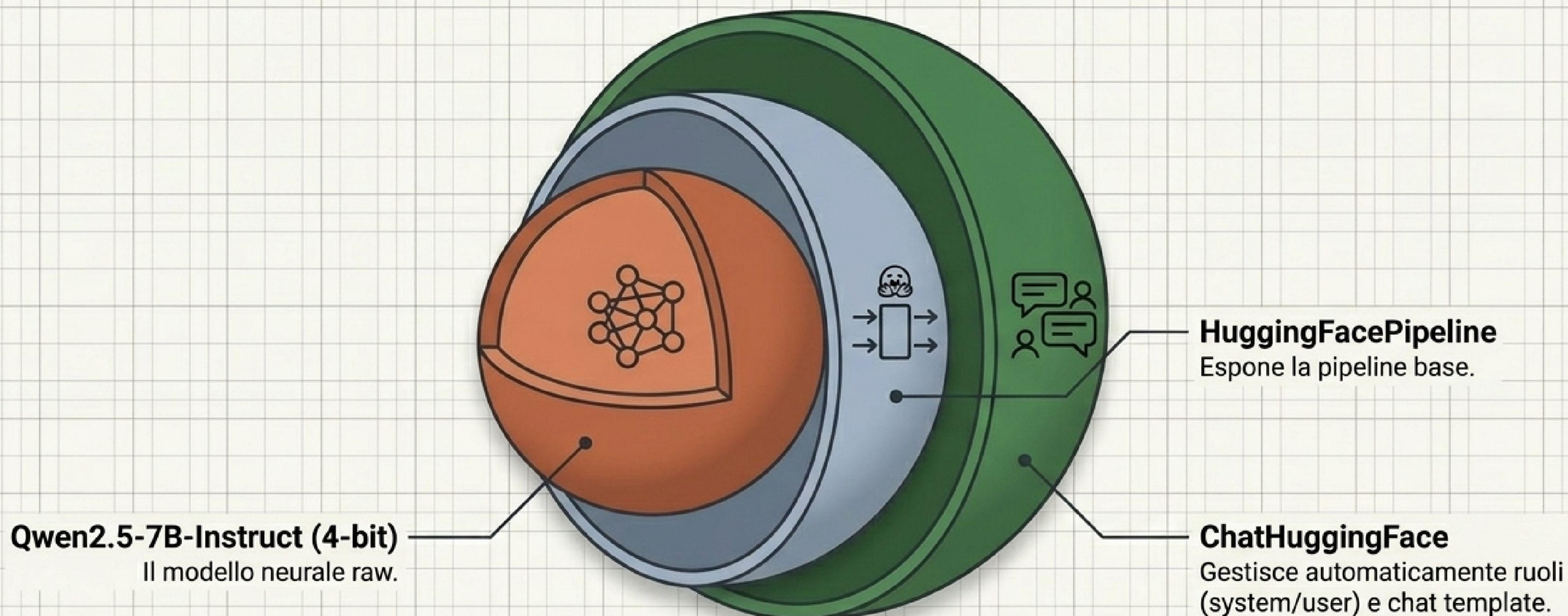
```
catena.invoke()
```

```
catena.batch([[...]])
```

```
catena.stream()
```



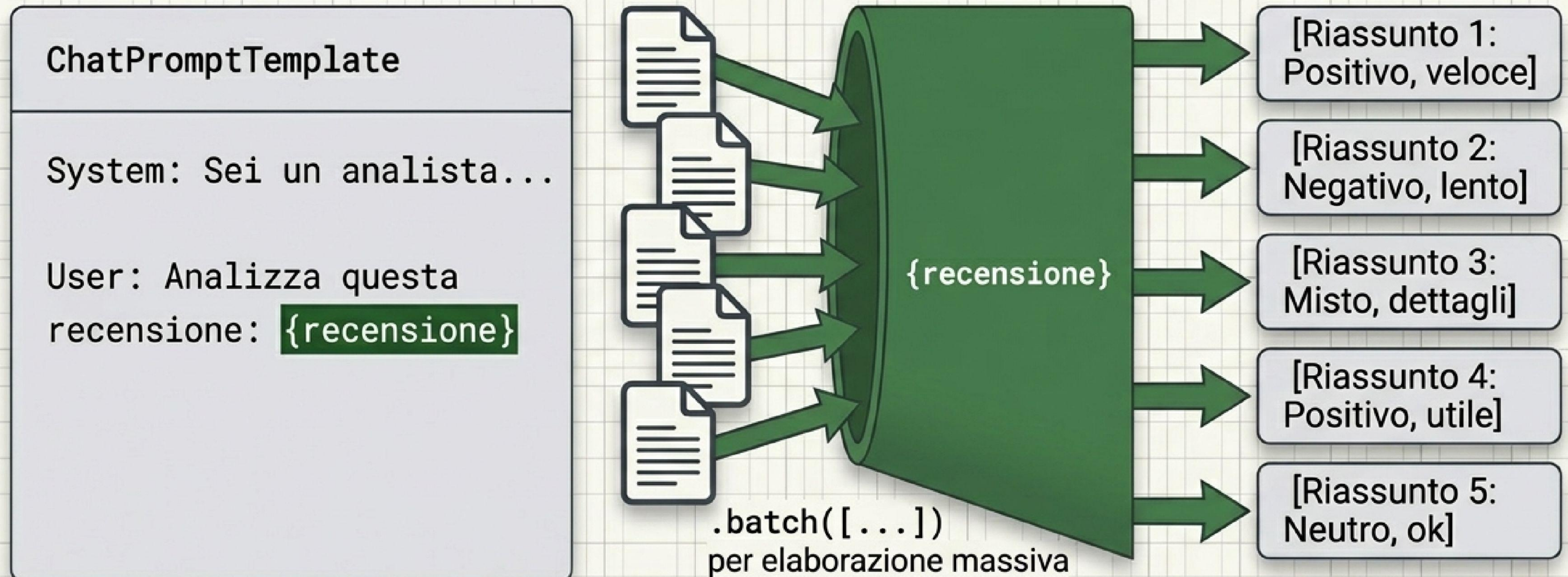
Avvolgere l'LLM: Creare l'Interfaccia Standard



➡ L'oggetto risultante è il motore pronto per essere inserito in qualsiasi catena LCEL.

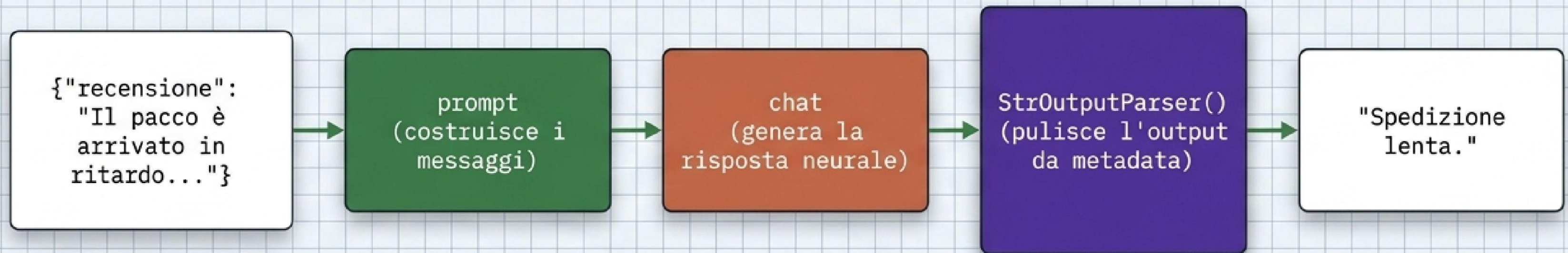
PromptTemplate: Scrivi una volta, esegui mille

Abbandoniamo le stringhe fisse. Usiamo ChatPromptTemplate per definire i ruoli e lasciare buchi per le variabili.



Assemblaggio: La Prima Catena LCEL

Uniamo i pezzi in una catena di base per estrarre una stringa pulita, nascondendo tutta la complessità.



Output Strutturato: Dall'Inerzia al JSON

Per integrare l'LLM nel software tradizionale serve un output strutturato, non prosa libera.

1. Schema (Pydantic)

Definiamo i campi attesi (sentiment, temi, riassunto).

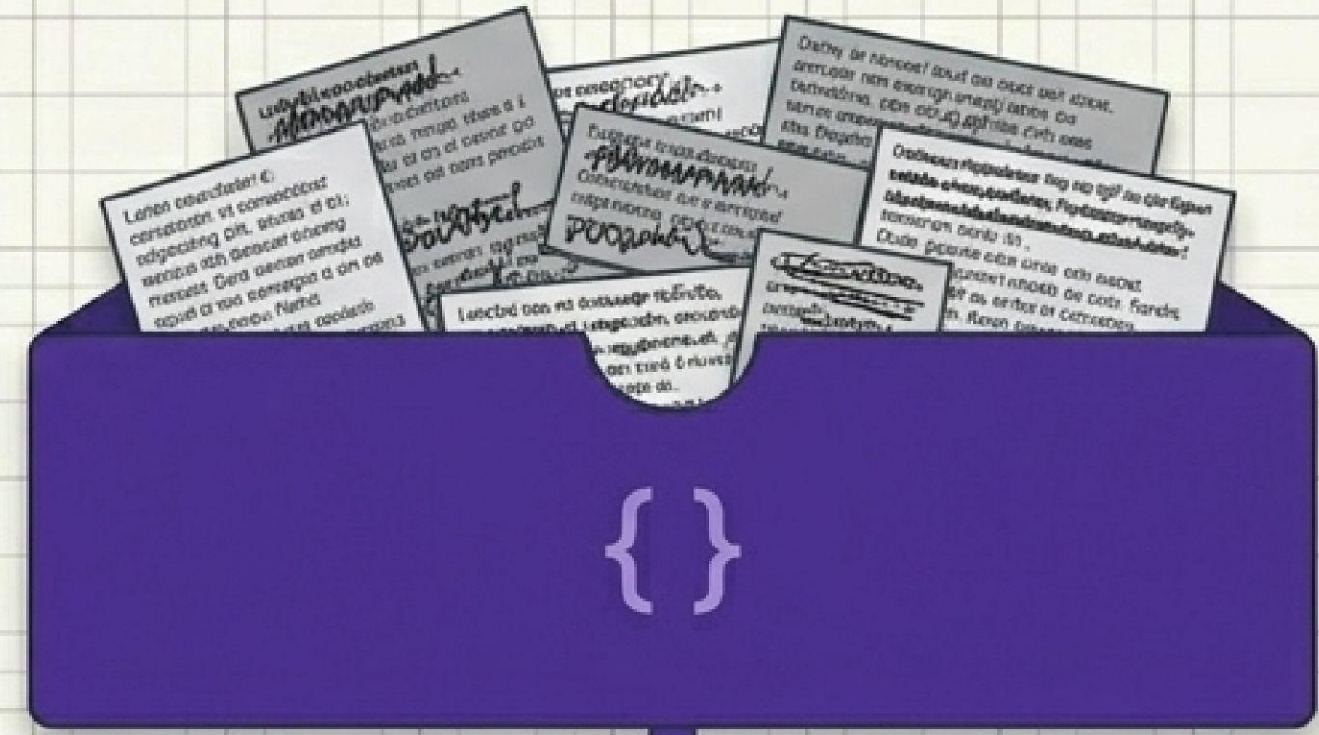
2. Istruzioni (JsonOutputParser)

Genera automaticamente le regole di formattazione per l'LLM e fa il parsing della risposta.

3. Esecuzione

`catena_json.invoke(...)`
restituisce un dizionario Python.

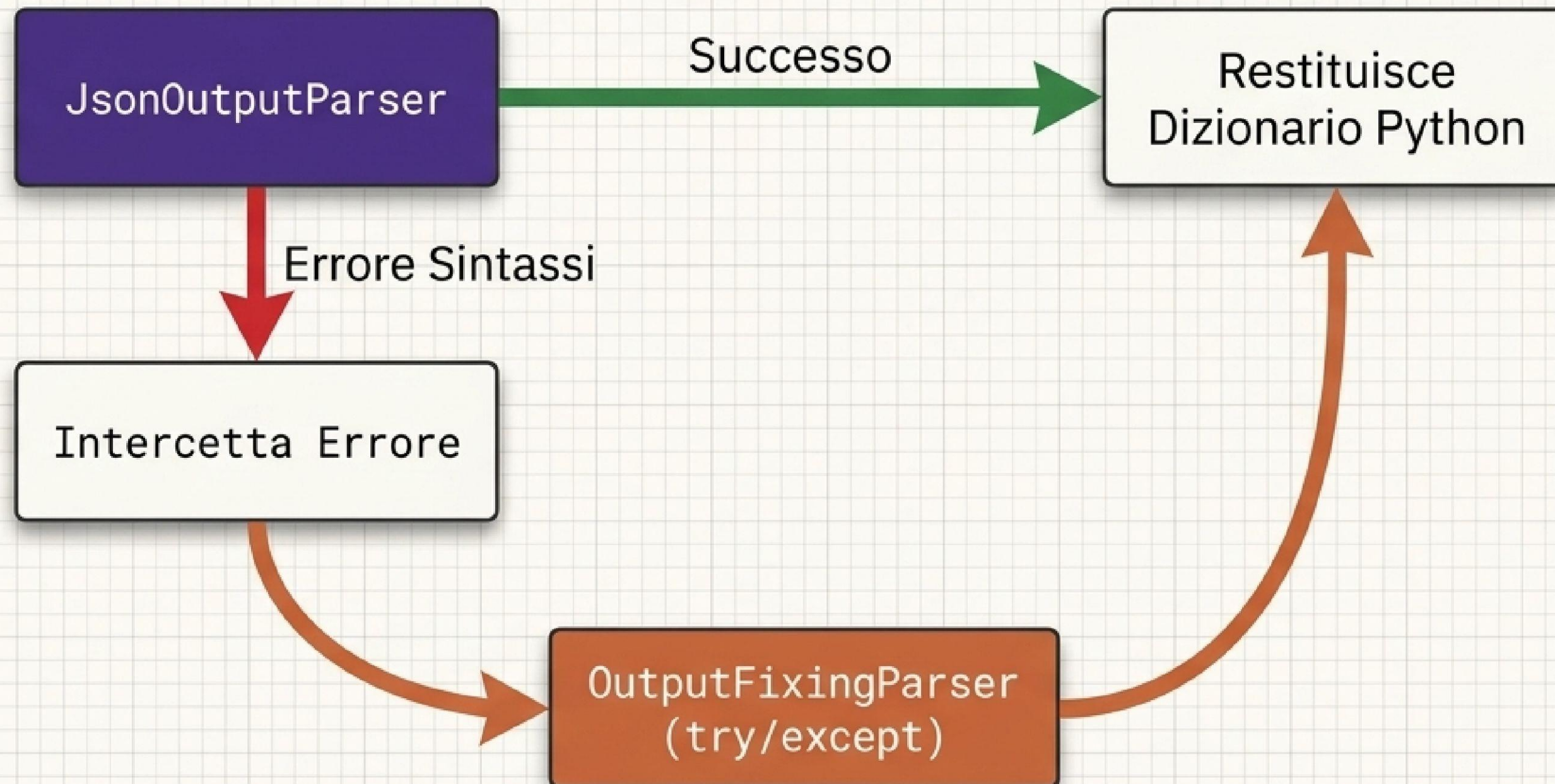
➔ I dati sono subito accessibili nel codice: `risultato['sentiment']`



```
{  
  "sentiment": "negativo",  
  "temi": ["spedizione"],  
  "riassunto": "..."  
}
```


Architetture Resilienti: Gestire il JSON Imperfetto ⚠

In produzione, modelli piccoli possono generare JSON non validi (es. una virgola di troppo). Servono strategie di fallback.

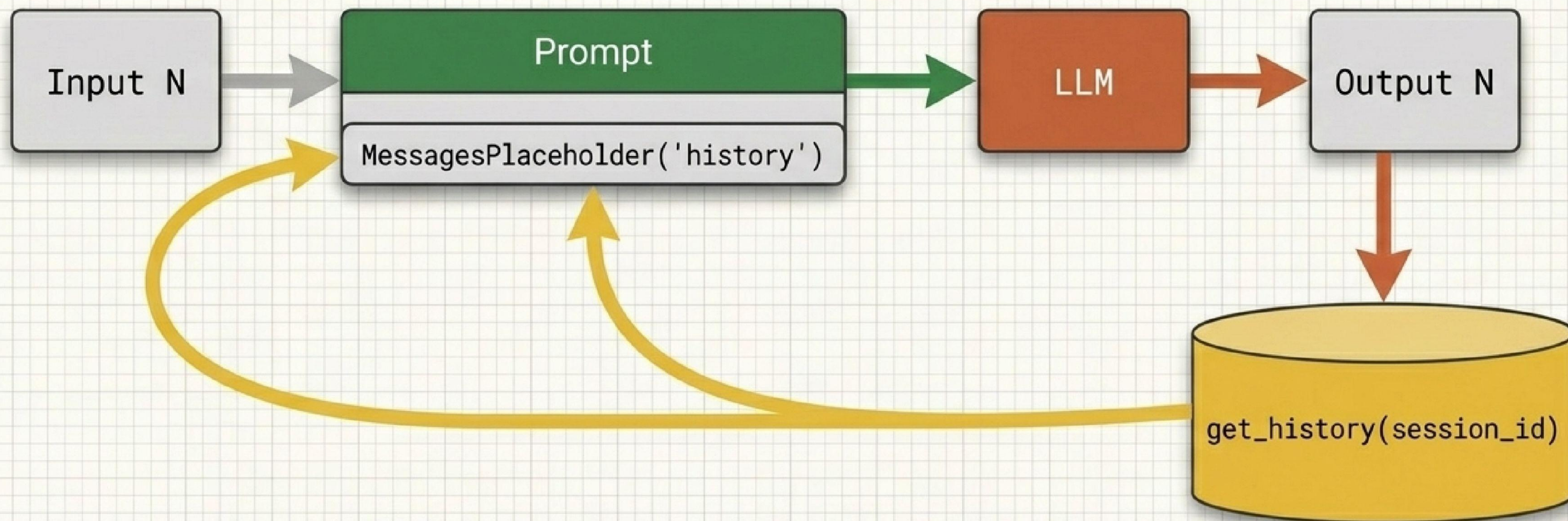


Pro-Tips

- Implementare try/except attorno all'invoke().
- Usare parser robusti integrati in LangChain.
- Abbassare la temperature per output più rigidi.

Memoria Conversazionale: Mantenere il Contesto

Di default, ogni `.invoke()` è amnesico. Per un chatbot serve un `RunnableWithMessageHistory` per iniettare dinamicamente lo storico.



La `session_id` è la chiave primaria: tiene separate le conversazioni di utenti diversi.

Demo: La Memoria in Azione

Verifichiamo il comportamento del modello iniettando e modificando i `session_id`.

`session_id = 'user_A'`

Turno 1:

User: Ciao, di cosa ti occupi?

AI: Mi presento, sono un analista di testi.

Turno 2:

User: Riassumi il tuo ruolo in 3 parole

AI: Analisi. Recensioni. Dati.

 Ricorda il contesto]

`session_id = 'user_B'`

Turno 1:

User: Riassumi il tuo ruolo in 3 parole

AI: Sono un'intelligenza artificiale generativa.

[Conversazione nuova e separata]

Hands-On: Modificare l'Architettura JSON

Esercizio: Estendere lo schema JSON esistente per far indovinare all'LLM il numero di stelle della recensione.

```
class AnalisiRecensione(BaseModel):  
    sentiment: str  
    temi: List[str]  
    valutazione_stimata: int = Field(description='Stima da 1 a 5 stelle')
```

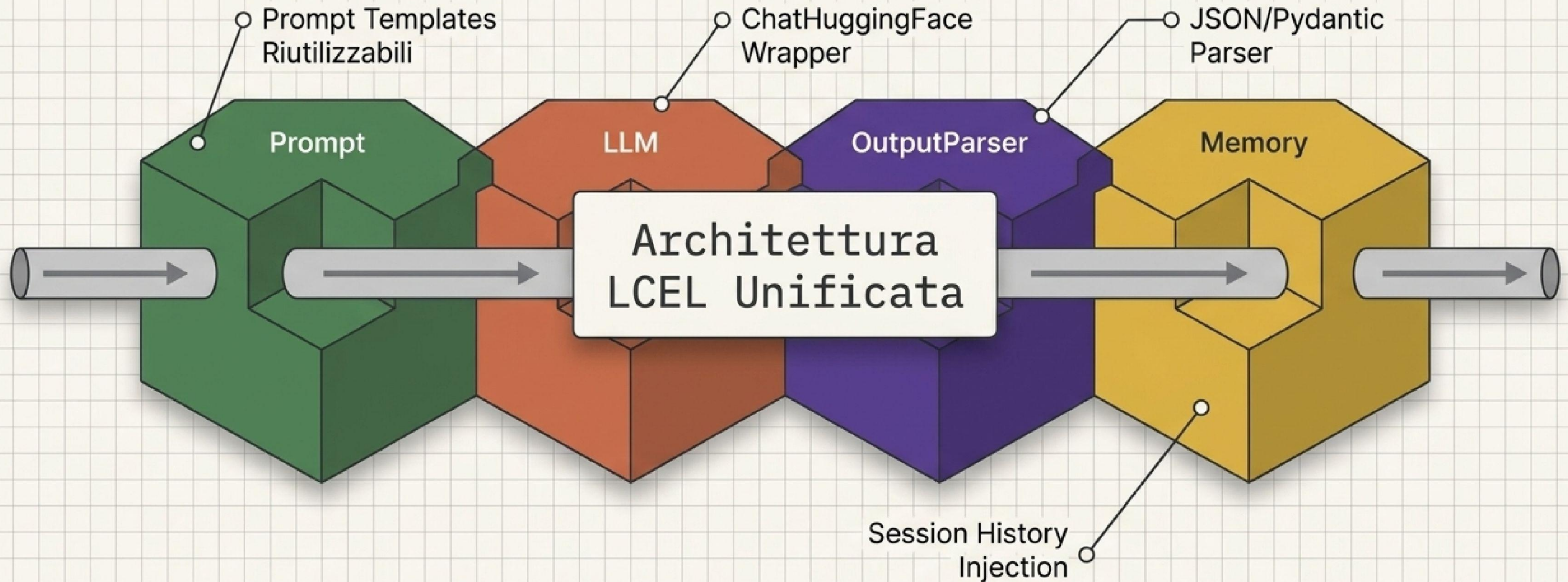
LLM Output JSON	Real Dataset
<code>{"valutazione_stimata": 2}</code>	Stars: 2



Variante Pro-Tip: Usa la stessa struttura per generare una bozza di risposta automatica dell'assistenza clienti!

Sintesi dell'Architettura

Abbiamo trasformato il modello grezzo in un'applicazione modulare, pronta per essere integrata.



Next Steps: Verso il RAG (Lezione 6)

Oggi: Dal modello all'applicazione ✅. **Prossimamente:** Uniamo gli Embeddings con le catene per creare un'app che risponde citando le fonti.

